1. Восстановление золота из руды

Необходимо подготовить прототип модели машинного обучения для «Цифры». Компания разрабатывает решения для эффективной работы промышленных предприятий.

Модель должна предсказать коэффициент восстановления золота из золотосодержащей руды. Используйте данные с параметрами добычи и очистки.

Модель поможет оптимизировать производство, чтобы не запускать предприятие с убыточными характеристиками.

Нам нужно:

1. Подготовить данные;
2. Провести исследовательский анализ данных;
3. Построить и обучить модель.

1.1. Подготовка данных

1.1.1. Импорты и чтение файлов

Ввод [1]:

```
import pandas as pd
from matplotlib import pyplot as plt
from sklearn.metrics import mean_absolute_error as mae
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import KFold
import warnings
warnings.filterwarnings("ignore")
```

Ввод [2]:

```
gr_train = pd.read_csv('/datasets/gold_recovery_train_new.csv')
gr_test = pd.read_csv('/datasets/gold_recovery_test_new.csv')
gr_full = pd.read_csv('/datasets/gold_recovery_full_new.csv')
```

1.1.2. Знакомство с данными

1.1.3. Расчет эффективности обогащения

Ввод [3]:

```
# для формулы (C * (F - T)) / (F * (C - T)) * 100
# создадим отдельные переменные, с соответствующими данными

C = gr_train['rougher.output.concentrate_au']
F = gr_train['rougher.input.feed_au']
T = gr_train['rougher.output.tail_au']
```

Ввод [4]:

```
# расчитанные по формуле значения сохраним в отдельную переменную
Recovery = (C * (F - T)) / (F * (C - T)) * 100
```

Ввод [5]:

```
# проверим отклонение по Mean Absolute Error
mae(Recovery, gr_train['rougher.output.recovery'])
```

Out[5]:

```
9.73512347450521e-15
```

Крайне малое значение, погрешностью которого (при согласии исследователей) можно пренебречь - Python и на более низких разрядах ошибается (особенно при делении нецелых чисел)

Ввод [6]:

```
# 0.3 / 0.1
```

1.1.4. Признаки тестовой выборки

Посмотрим на признаки набора, не вошедшие в тестовую выборку

Ввод [7]:

```
[col
 for col in gr_full.columns
 if col not in gr_test.columns]
```

Out[7]:

```
['final.output.concentrate_ag',
 'final.output.concentrate_pb',
 'final.output.concentrate_sol',
 'final.output.concentrate_au',
 'final.output.recovery',
 'final.output.tail_ag',
 'final.output.tail_pb',
 'final.output.tail_sol',
 'final.output.tail_au',
 'primary_cleaner.output.concentrate_ag',
 'primary_cleaner.output.concentrate_pb',
 'primary_cleaner.output.concentrate_sol',
 'primary_cleaner.output.concentrate_au',
 'primary_cleaner.output.tail_ag',
 'primary_cleaner.output.tail_pb',
 'primary_cleaner.output.tail_sol',
 'primary_cleaner.output.tail_au',
 'rougher.calculation.sulfate_to_au_concentrate',
 'rougher.calculation.floatbank10_sulfate_to_au_feed',
 'rougher.calculation.floatbank11_sulfate_to_au_feed',
 'rougher.calculation.au_pb_ratio',
 'rougher.output.concentrate_ag',
 'rougher.output.concentrate_pb',
 'rougher.output.concentrate_sol',
 'rougher.output.concentrate_au',
 'rougher.output.recovery',
 'rougher.output.tail_ag',
 'rougher.output.tail_pb',
 'rougher.output.tail_sol',
 'rougher.output.tail_au',
 'secondary_cleaner.output.tail_ag',
 'secondary_cleaner.output.tail_pb',
 'secondary_cleaner.output.tail_sol',
 'secondary_cleaner.output.tail_au']
```

В основном видны output - можно сделать вывод, что результативные данные подтягиваются позже, и мы, скорее всего, окажемся в ситуации, где у нас не будет этих данных при реальных предсказаниях

1.2. Анализ данных

1.2.1. Поэтапное изменение концентраций

Рассмотрим три объекта - концентрацию au, ag и pb. Также для наблюдения возьмем три этапа: черновой концентрат (после флотации), состояние после

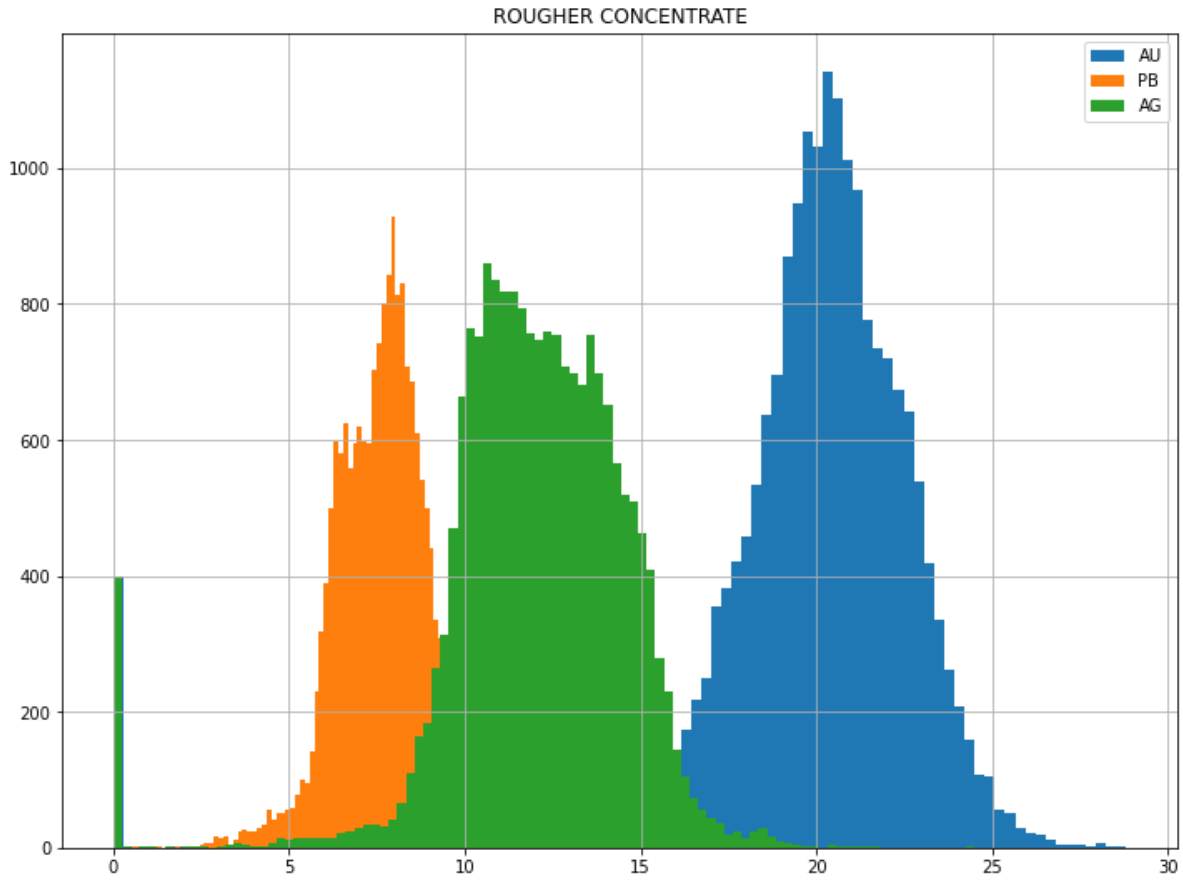
первичной очистки и финальное состояние (после вторичной очистки). Рассматривать будем распределения этих показателей

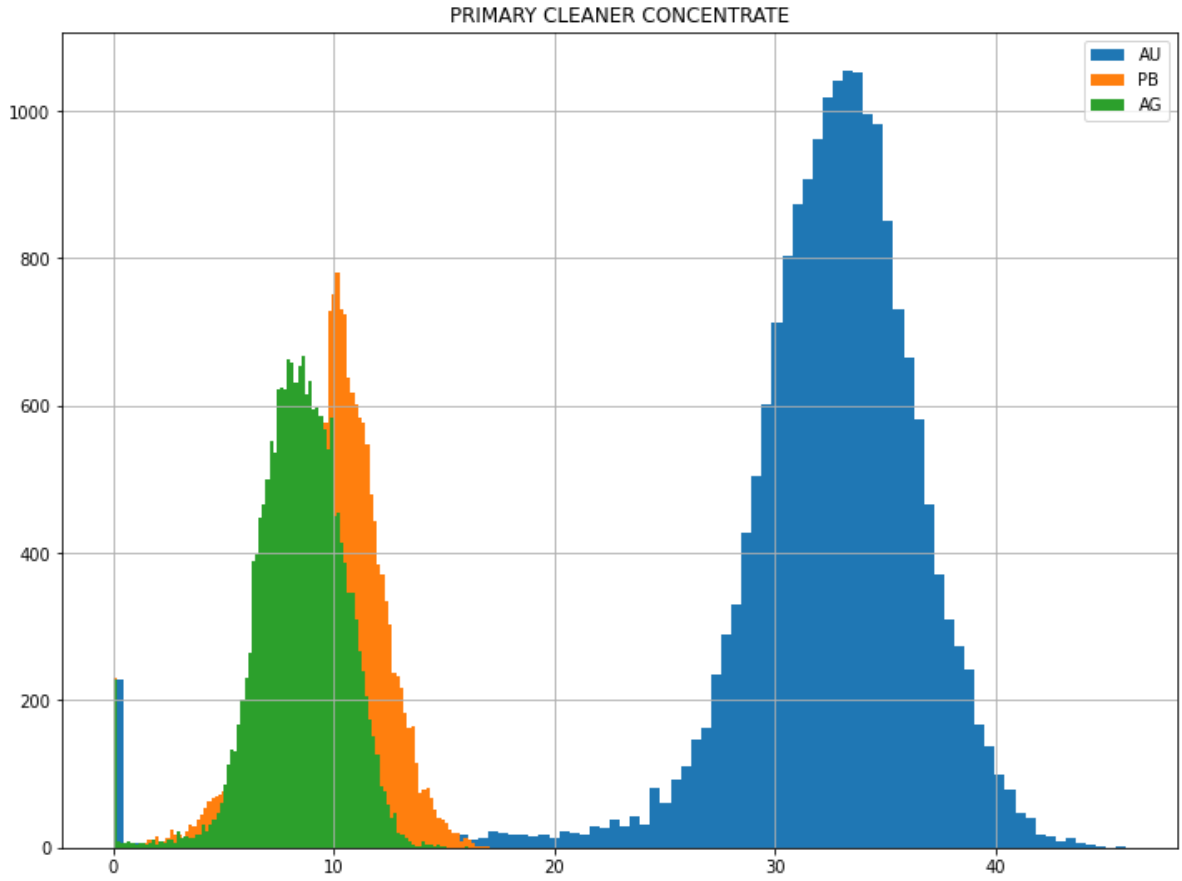
Ввод [8]:

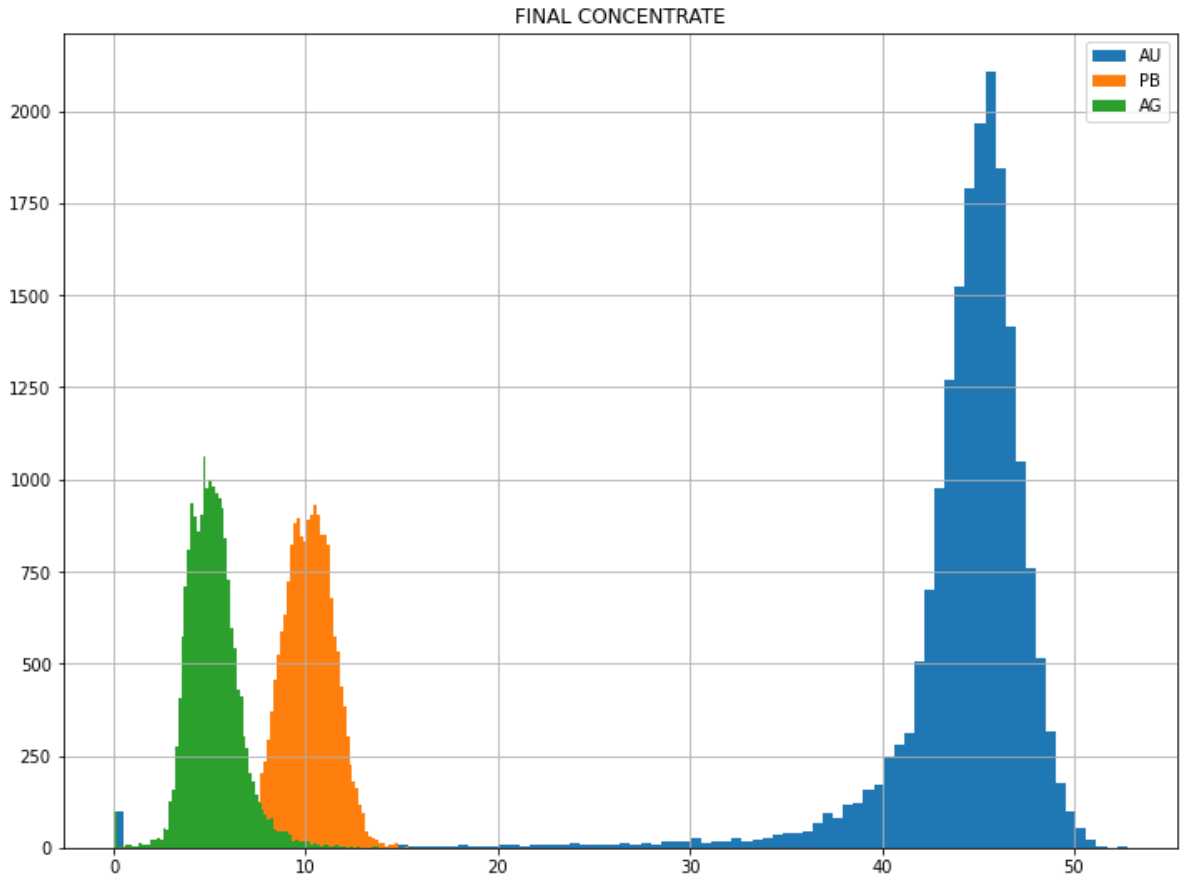
```
plt.figure(figsize=(12, 9))
gr_full['rougher.output.concentrate_au'].hist(bins=100, label='AU')
gr_full['rougher.output.concentrate_pb'].hist(bins=100, label='PB')
gr_full['rougher.output.concentrate_ag'].hist(bins=100, label='AG')
plt.title('ROUGHER CONCENTRATE')
plt.legend()
plt.show()

plt.figure(figsize=(12, 9))
gr_full['primary_cleaner.output.concentrate_au'].hist(bins=100, label='AU')
gr_full['primary_cleaner.output.concentrate_pb'].hist(bins=100, label='PB')
gr_full['primary_cleaner.output.concentrate_ag'].hist(bins=100, label='AG')
plt.title('PRIMARY CLEANER CONCENTRATE')
plt.legend()
plt.show()

plt.figure(figsize=(12, 9))
gr_full['final.output.concentrate_au'].hist(bins=100, label='AU')
gr_full['final.output.concentrate_pb'].hist(bins=100, label='PB')
gr_full['final.output.concentrate_ag'].hist(bins=100, label='AG')
plt.title('FINAL CONCENTRATE')
plt.legend()
plt.show()
```







Здесь видно, что au, концентрация которого нам в конечном итоге и важна, возрастает и в абсолютном выражении, и как концентрация, в частях

Значения pb остаются примерно на одном месте, ag, в свою очередь, незначительно уменьшается

1.2.2. Распределение размеров гранул сырья

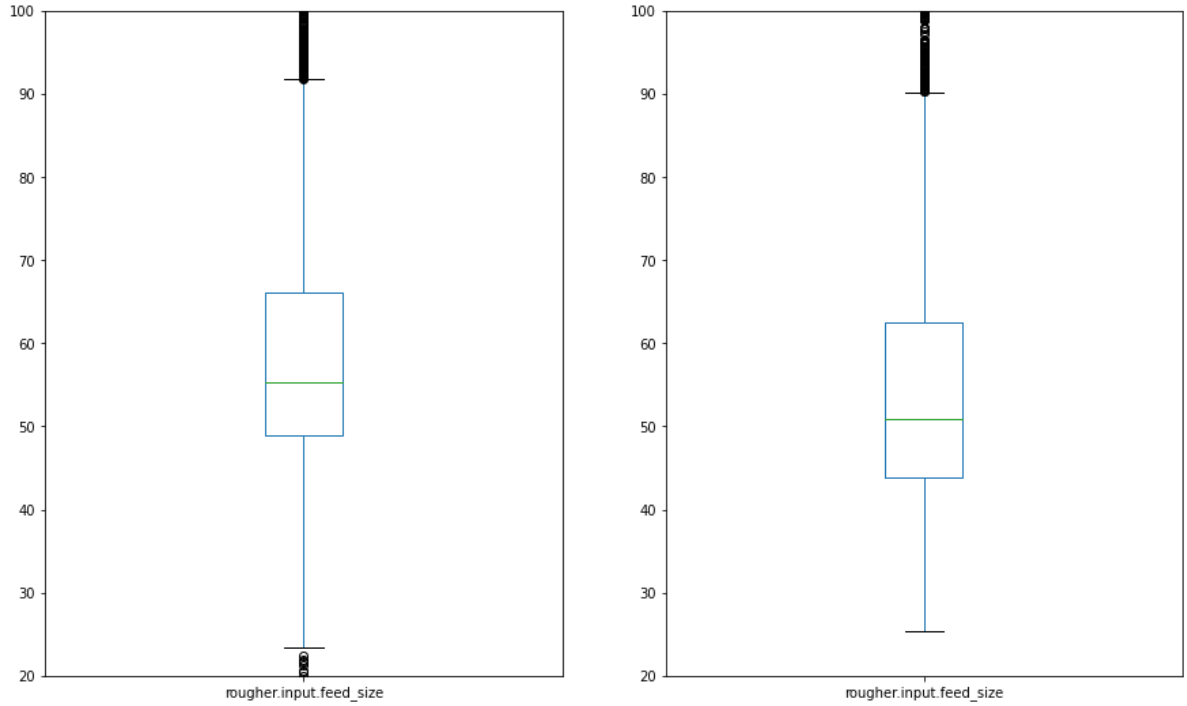
Построим boxplot

Ввод [9]:

```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(15, 9))
gr_train['rougher.input.feed_size'].plot(kind='box', ax=ax1, ylim=(20, 100))
gr_test['rougher.input.feed_size'].plot(kind='box', ax=ax2, ylim=(20, 100))
```

Out[9]:

```
<AxesSubplot:>
```



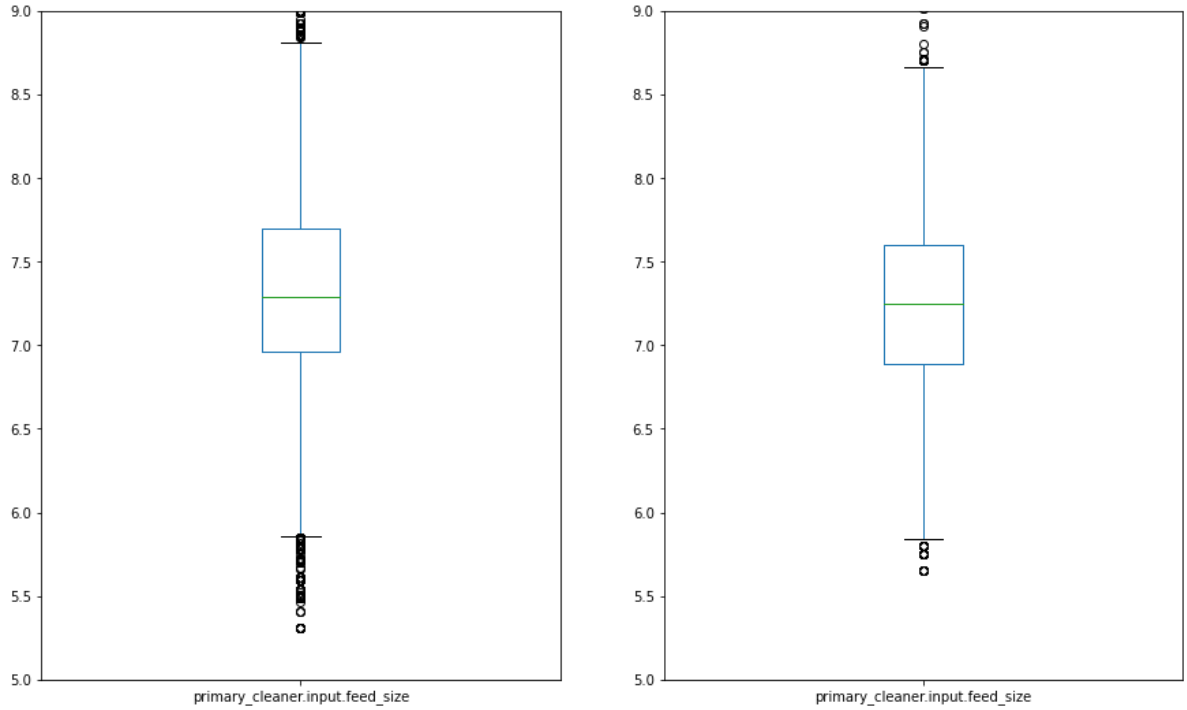
Распределения очень похожи, сильных отличий (в масштабах данных) не наблюдается

Ввод [10]:

```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(15, 9))
gr_train['primary_cleaner.input.feed_size'].plot(kind='box', ax=ax1, ylim=(5, 9))
gr_test['primary_cleaner.input.feed_size'].plot(kind='box', ax=ax2, ylim=(5, 9))
```

Out[10]:

```
<AxesSubplot:>
```



Здесь примерно та же история - выбросы иногда отличаются, но большая часть данных лежит в одинаковых пределах

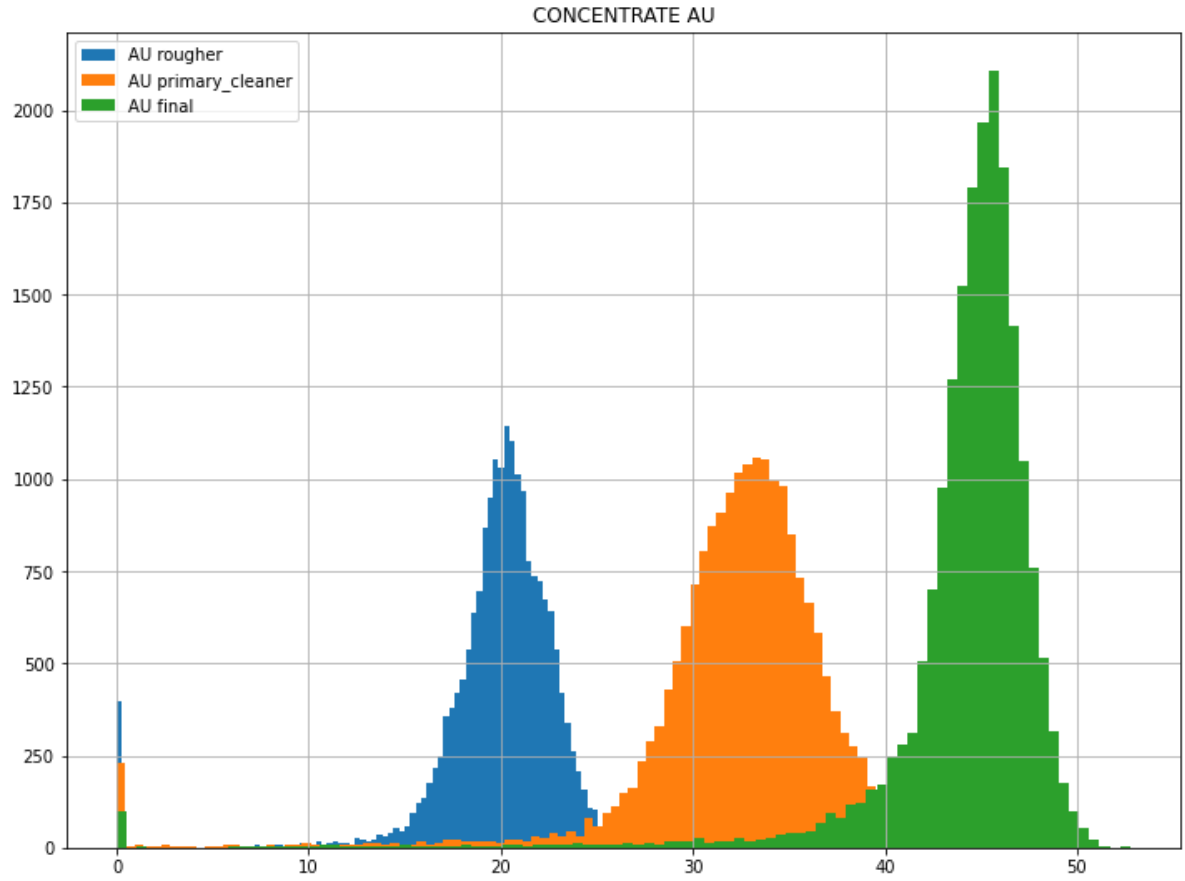
1.2.3. Концентрация на разных стадиях

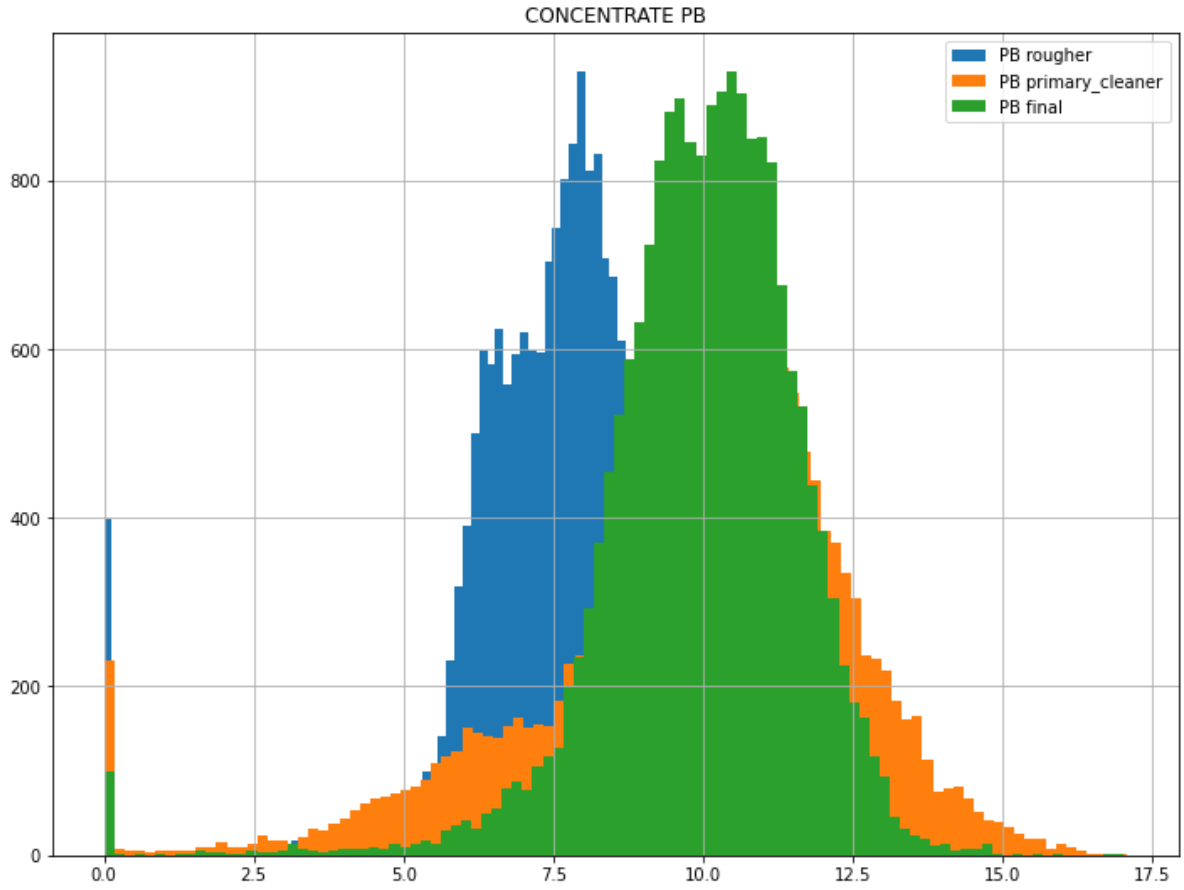
Можно посмотреть отдельно каждый элемент по этапам

Ввод [11]:

```
plt.figure(figsize=(12, 9))
gr_full['rougher.output.concentrate_au'].hist(bins=100, label='AU rougher')
gr_full['primary_cleaner.output.concentrate_au'].hist(bins=100, label='AU primary_cl
gr_full['final.output.concentrate_au'].hist(bins=100, label='AU final')
plt.title('CONCENTRATE AU')
plt.legend()
plt.show()

plt.figure(figsize=(12, 9))
gr_full['rougher.output.concentrate_pb'].hist(bins=100, label='PB rougher')
gr_full['primary_cleaner.output.concentrate_pb'].hist(bins=100, label='PB primary_cl
gr_full['final.output.concentrate_pb'].hist(bins=100, label='PB final')
plt.title('CONCENTRATE PB')
plt.legend()
plt.show()
```

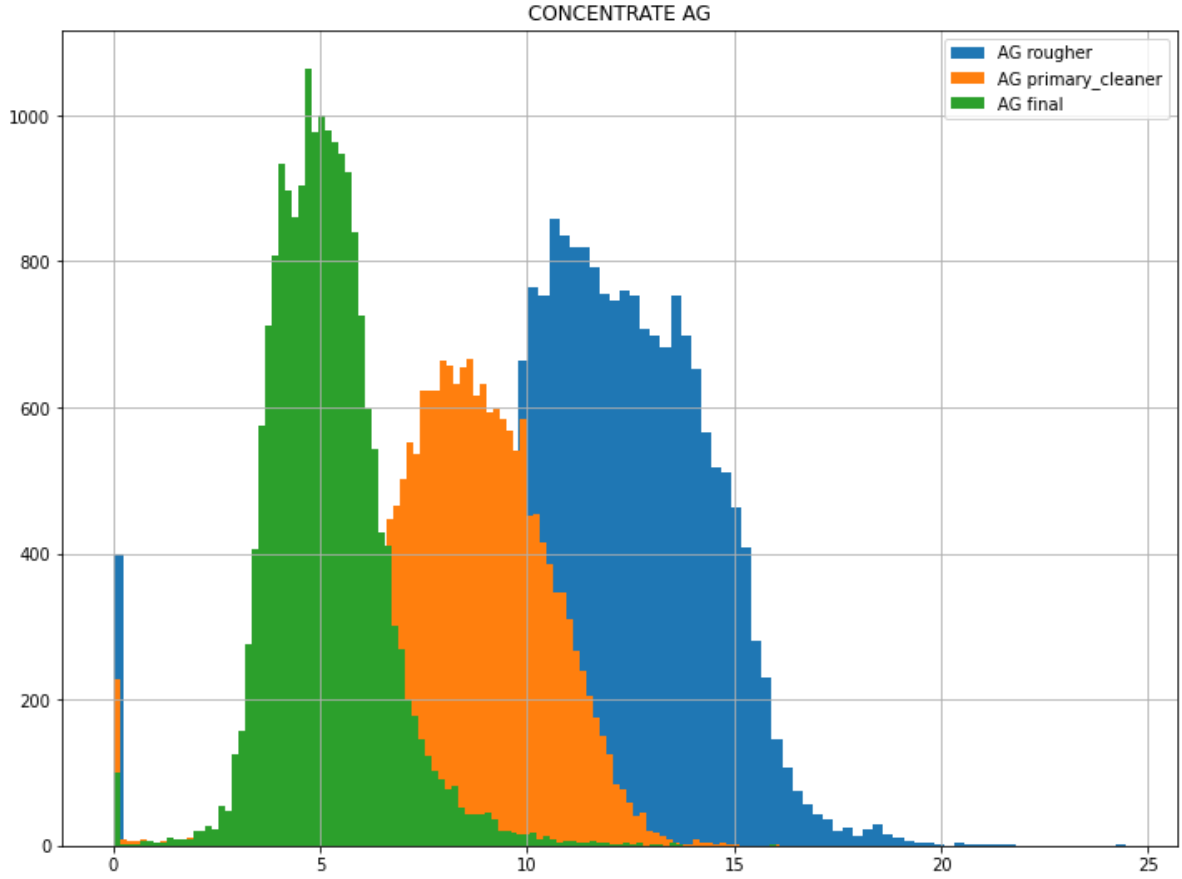


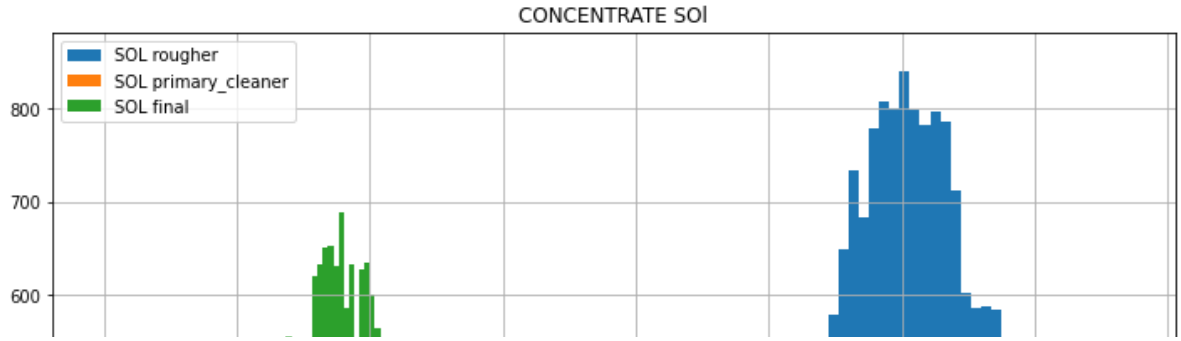

CONCENTRATE PB
PB rougher
PB primary_cleaner
PB final
0
200
400
600
800
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

Ввод [12]:

```
plt.figure(figsize=(12, 9))
gr_full['rougher.output.concentrate_ag'].hist(bins=100, label='AG rougher')
gr_full['primary_cleaner.output.concentrate_ag'].hist(bins=100, label='AG primary_cl
gr_full['final.output.concentrate_ag'].hist(bins=100, label='AG final')
plt.title('CONCENTRATE AG')
plt.legend()
plt.show()

plt.figure(figsize=(12, 9))
gr_full['rougher.output.concentrate_sol'].hist(bins=100, label='SOL rougher')
gr_full['primary_cleaner.output.concentrate_sol'].hist(bins=100, label='SOL primary_
gr_full['final.output.concentrate_sol'].hist(bins=100, label='SOL final')
plt.title('CONCENTRATE SOl')
plt.legend()
plt.show()
```





На графиках с уже одинаковым масштабом можно заметить резкий рост au на последней стадии, рост ag, практически неизменную концентрацию pb и значительный спад sol

Ввод [13]:

```
# для каждого этапа пропишем необходимые названия колонок концентрации
primary_cleaner_concentrate = ['primary_cleaner.output.concentrate_ag',
                               'primary_cleaner.output.concentrate_pb',
                               'primary_cleaner.output.concentrate_sol',
                               'primary_cleaner.output.concentrate_au']

final_concentrate = ['final.output.concentrate_ag',
                     'final.output.concentrate_pb',
                     'final.output.concentrate_sol',
                     'final.output.concentrate_au']

rougher_concentrate = ['rougher.output.concentrate_ag',
                       'rougher.output.concentrate_pb',
                       'rougher.output.concentrate_sol',
                       'rougher.output.concentrate_au']

# положим все в один словарь
full_concentrate = {'rougher': rougher_concentrate,
                    'primary': primary_cleaner_concentrate,
                    'final': final_concentrate}
```

Ввод [14]:

```
# создадим списки для хранения данных каждого вещества
ag, pb, sol, au, full = [], [], [], [], []

for step in full_concentrate:
    # сохраним средние данные по веществам
    concentrate = gr_full[full_concentrate[step]].mean()
    # выведем суммарную концентрацию всех веществ
    print(f'{step}: {concentrate.sum()}')
    # и каждого в отдельности
    print(f'ag: {round(concentrate[0], 2)} ({round(concentrate[0] / concentrate.sum(
          f'pb: {round(concentrate[1], 2)} ({round(concentrate[1] / concentrate.sum(
          f'sol: {round(concentrate[2], 2)} ({round(concentrate[2] / concentrate.sum
          f'au: {round(concentrate[3], 2)} ({round(concentrate[3] / concentrate.sum(

    # заполним списки
    ag.append(round(concentrate[0], 2))
    pb.append(round(concentrate[1], 2))
    sol.append(round(concentrate[2], 2))
    au.append(round(concentrate[3], 2))
    full.append(concentrate.sum())
```

```
rougher: 68.17860254823096
ag: 11.99 (17.59%)
pb: 7.61 (11.16%)
sol: 28.81 (42.25%)
au: 19.77 (29.0%)

primary: 60.88083772573897
ag: 8.44 (13.87%)
pb: 9.83 (16.15%)
sol: 10.49 (17.23%)
au: 32.12 (52.76%)

final: 68.72510242280688
ag: 5.17 (7.52%)
pb: 9.98 (14.52%)
sol: 9.5 (13.82%)
au: 44.08 (64.13%)
```

Здесь такая же картина - концентрация золота растет, остальные либо падают, либо остаются в пределах погрешности

1.3. Модель

1.3.1. sMAPE

Напишем функцию, вычисляющую sMAPE

Ввод [15]:

```
def sMAPE(target, predicted):
    N = len(target)
    y = pd.Series(target)
    y_ = pd.Series(predicted)
    numerator = abs(y - y_)
    denominator = (abs(y) + abs(y_)) / 2
    return (numerator / denominator).sum() / N * 100
```

На основе этой функции, напишем функцию, вычисляющую итоговое sMAPE

Ввод [16]:

```
def final_sMAPE(rougher_target, rougher_predicted, final_target, final_predicted):
    return 0.25 * sMAPE(rougher_target, rougher_predicted) + 0.75 * sMAPE(final_targ
```

1.3.2. Преподготовка

Так как в тестовой выборке отсутствуют целевые признаки, для оценки работы лучшей модели необходимо их туда подтянуть

Ввод [17]:

```
gr_test_added = (gr_test
                 .merge(gr_full[['date', 'rougher.output.recovery']], on='date')
                 .merge(gr_full[['date', 'final.output.recovery']], on='date'))
```

Ввод [18]:

```
gr_test_added.head()
```

Out[18]:

	date	primary_cleaner.input.sulfate	primary_cleaner.input.depressant	primary_cleaner.input.fe
0	2016-09-01 00:59:59	210.800909	14.993118	8
1	2016-09-01 01:59:59	215.392455	14.987471	8
2	2016-09-01 02:59:59	215.259946	12.884934	7
3	2016-09-01 03:59:59	215.336236	12.006805	7
4	2016-09-01 04:59:59	199.099327	10.682530	7

5 rows × 55 columns

Также пропишем функцию для кросс-валидации

Ввод [19]:

```
def cross_val(model, X, y, cv=3):
    scores = []
    kf = KFold(n_splits=cv)
    kf.get_n_splits(y)
    for train_index, test_index in kf.split(y):
        X_train, X_test = X.loc[train_index], X.loc[test_index]
        y_train, y_test = y.loc[train_index], y.loc[test_index]
        model.fit(X_train, y_train)
        scores.append(sMAPE(y_test, model.predict(X_test)))
    return pd.Series(scores).mean()
```

1.3.3. Прогнозирование

1.3.3.1. Rougher Output Recovery

1.3.3.1.1. Анализ колонок

На первом шаге нас интересует rougher.output.recovery - результат флотации. Таким образом, чтобы предсказывать его, нам доступны только данные, идущие ДО этого этапа:

- 'rougher.calculation.sulfate_to_au_concentrate'

- 'rougher.calculation.floatbank10_sulfate_to_au_feed'

- 'rougher.calculation.floatbank11_sulfate_to_au_feed'

- 'rougher.calculation.au_pb_ratio'

- 'rougher.input.feed_ag'

- 'rougher.input.feed_pb'

- 'rougher.input.feed_rate'

- 'rougher.input.feed_size'

- 'rougher.input.feed_sol'

- 'rougher.input.feed_au'

- 'rougher.input.floatbank10_sulfate'

- 'rougher.input.floatbank10_xanthate'

- 'rougher.input.floatbank11_sulfate'

- 'rougher.input.floatbank11_xanthate'

- 'rougher.output.concentrate_ag'

- 'rougher.output.concentrate_pb'

- 'rougher.output.concentrate_sol'

- 'rougher.output.concentrate_au'

Немного очистив данные, выкинув результаты других показателей, а также рассчеты, получим следующий набор признаков:

- 'rougher.input.feed_ag'

- 'rougher.input.feed_pb'

- 'rougher.input.feed_rate'

- 'rougher.input.feed_size'

- 'rougher.input.feed_sol'

- 'rougher.input.feed_au'

- 'rougher.input.floatbank10_sulfate'

- 'rougher.input.floatbank10_xanthate'

- 'rougher.input.floatbank11_sulfate'

- 'rougher.input.floatbank11_xanthate'

Собственно, все, что подается на вход

Оставим только те колонки, которые есть в тестовой выборке

Ввод [20]:

```
train_columns_rougher = ['rougher.input.feed_ag', 'rougher.input.feed_pb',
                         'rougher.input.feed_rate', 'rougher.input.feed_size',
                         'rougher.input.feed_sol', 'rougher.input.feed_au',
                         'rougher.input.floatbank10_sulfate', 'rougher.input.floatba
                         'rougher.input.floatbank11_sulfate', 'rougher.input.floatba

result_columns_rougher = []
for column in train_columns_rougher:
    if column in gr_test:
        result_columns_rougher.append(column)
    else:
        print(column)
```

Собственно, остались все, но лишним такой маневр не будет

Ввод [21]:

```
gr_train = gr_train.dropna().reset_index(drop=True)
gr_train_features_rougher = gr_train[result_columns_rougher]
gr_train_target_rougher = gr_train['rougher.output.recovery']
```

Так как данные указывают на регрессию, будем использовать четыре основных регрессора:

- LinearRegression

- DecisionTreeRegressor

- RandomForestRegressor

LinearRegression

Ввод [22]:

```
best_Linear_model_rougher = LinearRegression()
print(cross_val(best_Linear_model_rougher, gr_train_features_rougher, gr_train_targe
```

```
3.399688153402923
```

Разброс в 3.4% - не самый большой, который мы могли иметь

DecisionTreeRegressor

Ввод [23]:

```
best_dtr_model = None
best_dtr_sMAPE = False
best_dtr_depth = 0

for depth in range(1, 21):
    model = DecisionTreeRegressor(max_depth=depth, random_state=12345)
    dtr_sMAPE = cross_val(model, gr_train_features_rougher, gr_train_target_rougher,
    if not best_dtr_sMAPE:
        best_dtr_sMAPE = dtr_sMAPE
    if dtr_sMAPE < best_dtr_sMAPE:
        best_dtr_model = model
        best_dtr_sMAPE = dtr_sMAPE
        best_dtr_depth = depth

best_dtr_sMAPE, best_dtr_depth
```

Out[23]:

```
(3.254858242329988, 2)
```

Здесь разброс чуть ближе к 3.25%, и всего 2 уровня, следовательно - модель плохо подходит для данной задачи

RandomForestRegressor

Ввод [24]:

```
best_rfr_model = None
best_rfr_sMAPE = False
best_rfr_depth = 0

for depth in range(1, 6):
    for estimators in range(95, 106, 5):
        model = RandomForestRegressor(n_estimators=estimators,
                                      max_depth=depth,
                                      random_state=12345)
        rfr_sMAPE = cross_val(model, gr_train_features_rougher, gr_train_target_roug
        if not best_rfr_sMAPE:
            best_rfr_sMAPE = rfr_sMAPE
        if rfr_sMAPE < best_rfr_sMAPE:
            best_rfr_model = model
            best_rfr_sMAPE = rfr_sMAPE
            best_rfr_depth = depth
            best_rfr_estimators = estimators

best_rfr_sMAPE, best_rfr_depth, best_rfr_estimators
```

Out[24]:

```
(3.3777171547708815, 5, 100)
```

Здесь примерно так же - 3.4% отклонений и 5 уровней

1.3.3.2. Final Output Recovery

1.3.3.2.1. Анализ колонок

Так как это финальная стадия, добавим к уже существующим колонкам еще несколько:

- 'primary_cleaner.input.sulfate'

- 'primary_cleaner.input.depressant',

- 'primary_cleaner.input.feed_size'

- 'primary_cleaner.input.xanthate'

Ввод [25]:

```
train_columns_final = train_columns_rougher + ['primary_cleaner.input.sulfate',
                                               'primary_cleaner.input.depressant',
                                               'primary_cleaner.input.feed_size',
                                               'primary_cleaner.input.xanthate']

result_columns_final = []

for column in train_columns_final:
    if column in gr_test:
        result_columns_final.append(column)
    else:
        print(column)
```

Все значения вошли

Ввод [26]:

```
gr_train_features_final = gr_train[result_columns_final]
gr_train_target_final = gr_train['final.output.recovery']
```

1.3.3.2.2. LinearRegression

Ввод [27]:

```
best_Linear_model_final = LinearRegression()
print(cross_val(best_Linear_model_final, gr_train_features_final, gr_train_target_fi
```

```
3.1033350486199325
```

Значительно ниже - около 3%

1.3.3.2.3. DecisionTreeRegressor

Ввод [28]:

```
best_dtr_model = None
best_dtr_sMAPE = False
best_dtr_depth = 0

for depth in range(1, 21):
    model = DecisionTreeRegressor(max_depth=depth, random_state=12345)
    dtr_sMAPE = cross_val(model, gr_train_features_final, gr_train_target_final, 3)
    if not best_dtr_sMAPE:
        best_dtr_sMAPE = dtr_sMAPE
    if dtr_sMAPE < best_dtr_sMAPE:
        best_dtr_model = model
        best_dtr_sMAPE = dtr_sMAPE
        best_dtr_depth = depth

best_dtr_sMAPE, best_dtr_depth
```

Out[28]:

```
(2.8392696825809485, 0)
```

Разброс такой же небольшой, однако видно, что модель отвечала наугад - 0 уровней

1.3.3.2.4. RandomForestRegressor

Ввод [29]:

```
best_rfr_model = None
best_rfr_sMAPE = False
best_rfr_depth = 0

for depth in range(1, 6):
    for estimators in range(95, 106, 5):
        model = RandomForestRegressor(n_estimators=estimators,
                                      max_depth=depth,
                                      random_state=12345)
        rfr_sMAPE = cross_val(model, gr_train_features_final, gr_train_target_final,
        if not best_rfr_sMAPE:
            best_rfr_sMAPE = rfr_sMAPE
        if rfr_sMAPE < best_rfr_sMAPE:
            best_rfr_model = model
            best_rfr_sMAPE = rfr_sMAPE
            best_rfr_depth = depth
            best_rfr_estimators = estimators

best_rfr_sMAPE, best_rfr_depth, best_rfr_estimators
```

Out[29]:

```
(2.7958333865444565, 2, 105)
```

Прогресс - четыре уровня, почти 3%

Напишем функцию, которая будет вычислять результативный sMAPE, обучая лучшую модель на обоих датасетах

Ввод [30]:

```
def result_modeling(best_model,
                    rougher_features, rougher_target,
                    final_features, final_target,
                    hyper=()):

    rougher_model = best_model(*hyper)
    rougher_sMAPE = cross_val(model, rougher_features.reset_index(drop=True),
                              rougher_target.reset_index(drop=True), 3)

    final_model = best_model(*hyper)
    final_sMAPE = cross_val(model, final_features.reset_index(drop=True),
                            final_target.reset_index(drop=True), 3)

    return 0.25 * rougher_sMAPE + 0.75 * final_sMAPE
```

Ввод [31]:

```
result_modeling(LinearRegression,
                gr_train_features_rougher, gr_train_target_rougher,
                gr_train_features_final, gr_train_target_final)
```

Out[31]:

```
3.0302804415838187
```

1.3.4. Тестирование

1.3.4.1. Разделение данных

Ввод [32]:

```
gr_test_added = gr_test_added.dropna().reset_index(drop=True)

# данные для обучения
gr_test_train_final = gr_test_added[result_columns_final]
gr_test_train_rougher = gr_test_added[result_columns_rougher]

# данные для проверки
gr_test_target_rougher = gr_test_added['rougher.output.recovery']
gr_test_target_final = gr_test_added['final.output.recovery']
```

1.3.4.2. Final sMAPE

Во всех тестах лучше всех себя показала линейная регрессия - ей и будем пользоваться

Ввод [33]:

```
# две обученные лучшие модели заставляем предсказывать на данных из тестовой выборки

final_predicted = best_Linear_model_final.predict(gr_test_train_final)
rougher_predicted = best_Linear_model_rougher.predict(gr_test_train_rougher)
```

Ввод [34]:

```
final_sMAPE(gr_test_target_rougher, rougher_predicted,
            gr_test_target_final, final_predicted)
```

Out[34]:

```
9.906901621181206
```

Итог: лучше всего под задачу подходит модель линейной регрессии, разброс составляет 9.9% на тестовой выборке